DSA Interview Guide: Quick Reference

Q1. What is Big-O?

Big-O describes how the time or space required by an algorithm grows as the input size increases.

Example:

If we search every element in an array:

[10, 20, 30, 40, 50]

We may have to check all n elements.

So the time complexity is:

O(n)

If we directly access arr[3], it takes constant time:

O(1)

Q2. What is the difference between Big-O, Big-Theta, and Big-Omega?

Big-O: Asymptotic upper bound. Big-Omega: Asymptotic lower bound. Big-Theta: Tight asymptotic bound.

Example:

Suppose an algorithm always scans all n elements.

Its complexity can be described as:

O(n)
Ω(n)
Θ(n)

because its growth is tightly linear.

Q3. What is the difference between time complexity and space complexity?

Time complexity describes running time growth; space complexity describes additional memory needs as input size increases.

Example:

for (int i = 0; i < n; i++) {

```
    System.out.println(arr[i]);
}
```

Time:

O(n)

If we don't create additional data structures, the **auxiliary space** is:

O(1)

Q4. What is the time complexity of array access?

Accessing an element by index in an array is O(1) because the address is calculated directly.

O(1)

Example:

arr[5]

We don't need to search through the previous elements.

Q5. What is the time complexity of HashMap lookup?

HashMap lookup is O(1) average-case, but can degrade to O(n) in the worst case due to collisions.

Example:

map.get("Rahul");

The hash of "Rahul" helps locate the corresponding entry quickly.

Q6. What is the time complexity of binary search?

Binary search on a sorted collection has a time complexity of O(log n) by repeatedly halving the search space.

Example:

[10, 20, 30, 40, 50, 60, 70]

To find 60, we don't check every element.

We repeatedly divide the search range:

7 elements

↓
3–4 elements
↓
1–2 elements
↓
found

Q7. What are the complexities of common sorting algorithms?

The efficiency varies by algorithm, with Merge, Quick (average), and Heap sort being the most scalable.

Algorithm	Average Time	Worst Time
Bubble Sort	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n^2)$	$O(n^2)$
Merge Sort	O(n log n)	O(n log n)
Quick Sort	O(n log n)	$O(n^2)$
Heap Sort	O(n log n)	O(n log n)

Example:

Q8. What is the time complexity of stack operations?

Stack operations (push, pop, peek) are all O(1) because they occur at the top.

push() → O(1)
pop() → O(1)
peek() → O(1)

Example:

Stack:

10
20
30 ← top

pop() removes 30 directly, so we don't need to search through the stack.

Q9. What is the time complexity of queue operations?

Queue operations (enqueue, dequeue, front) are O(1) in a standard implementation.

enqueue() → O(1)
dequeue() → O(1)
front() → O(1)

Example:

Front
↓
10 → 20 → 30
↑
Rear

Removing 10 from the front takes constant time.

Q10. How would you solve Two Sum?

Use a HashMap to store values as you scan; for each number, check if (target - current) exists in the map for O(n) time.

Example:

Array = [2, 7, 11, 15]
Target = 9

Start with 2.

Need:

9 - 2 = 7

Later we find 7.

So:

2 + 7 = 9

Q11. What is the maximum subarray problem?

Find the contiguous subarray with the largest sum using Kadane's Algorithm in O(n) time.

Example:

[-2, 1, -3, 4, -1, 2, 1, -5, 4]

The maximum-sum subarray is:

[4, -1, 2, 1]

Sum:

6

Q12. How do you move all zeroes to the end of an array?

Use a two-pointer technique to swap non-zero elements forward in O(n) time and O(1) space.

Example:

Input:
[0, 1, 0, 3, 12]

Output:
[1, 3, 12, 0, 0]

Time:

O(n)

Space:

O(1)

if we modify the array in place.

Q13. How do you remove duplicates from a sorted array?

Use two pointers to track unique elements in-place for O(n) time and O(1) space.

Example:

Input:
[1, 1, 2, 2, 3]

Output:
[1, 2, 3]

Time:

O(n)

Space:

O(1)

for an in-place solution.

Q14. How do you rotate an array?

Apply the three-reversal method (reverse parts and then the whole) for O(n) time and O(1) space.

Example:

[1,2,3,4,5]

Rotate right by 2:

[4,5,1,2,3]

Time:

O(n)

Space:

O(1)

Q15. How do you merge two sorted arrays?

Use two pointers to compare and merge elements into a new result array in O(n + m) time.

Example:

A = [1, 3, 5]
B = [2, 4, 6]

Result = [1, 2, 3, 4, 5, 6]

Time:

O(n + m)

where n and m are the lengths of the two arrays.

Q16. How do you find the longest substring without repeating characters?

Use a sliding window with a HashSet to maintain unique characters for O(n) time.

Example:

String = "abcabcbb"

The longest substring without repeating characters is:

"abc"

Length:

3

Time:

O(n)

Q17. How do you check whether two strings are anagrams?

Count character frequencies using an array or map; if they match exactly, the strings are anagrams (O(n)).

Example:

"listen"
"silent"

Both contain the same characters with the same frequencies.

Therefore:

They are anagrams.

Q18. What is frequency counting?

Frequency counting stores occurrences of each value (often using a HashMap) to solve grouping/duplicate problems.

Example:

Array = [1, 2, 2, 3, 3, 3]

Frequency:

1 → 1
2 → 2
3 → 3

Q19. How do you reverse a linked list?

Iterate through the list using three pointers (prev, current, next) to reverse directions in O(n) time.

prev
current
next

Example:

Before:
1 → 2 → 3 → null

After:
3 → 2 → 1 → null

Time:

O(n)

Space:

O(1)

for the iterative approach.

Q20. How do you detect a cycle in a linked list?

Use Floyd's Cycle Detection (slow and fast pointers); if they meet, a cycle exists (O(n) time, O(1) space).

Example:

1 → 2 → 3 → 4
↑ ↓
← ← ←

Time:

O(n)

Space:

O(1)

Q21. How do you find the middle node of a linked list?

Use slow and fast pointers; when fast reaches the end, slow is at the middle (O(n)).

slow → moves 1 step
fast → moves 2 steps

Example:

1 → 2 → 3 → 4 → 5
↑
slow

Middle = 3.

Time:

O(n)

Space:

O(1)

Q22. How do you remove the Nth node from the end?

Maintain a gap of N between two pointers; move them together until fast reaches the end to locate the target node (O(n)).

Example:

1 → 2 → 3 → 4 → 5

Remove the 2nd node from the end:

1 → 2 → 3 → 5

Time:

O(n)

Space:

O(1)

Q23. How do you merge two sorted linked lists?

Repeatedly attach the smaller of two current nodes to a result list using pointers (O(n+m)).

Example:

List 1: 1 → 3 → 5
List 2: 2 → 4 → 6

Result:

1 → 2 → 3 → 4 → 5 → 6

Time:

O(n + m)

Q24. How do you find the intersection of two linked lists?

Redirect each pointer to the other list's head after finishing; they will meet at the intersection in O(n+m) time.

Example:

A: 1 → 2
↘
7 → 8
↗
B: 4 → 5

The intersection starts at:

7

Time:

O(n + m)

Space:

O(1)

Q25. How would you implement a stack?

Implement using an array or linked list following LIFO (Last In, First Out) for O(1) operations.

Example:

push(10)
push(20)
push(30)

Stack:
30 ← top
20
10

Calling pop() removes 30.

Q26. How would you implement a queue?

Implement using a linked list or circular array following FIFO (First In, First Out) for O(1) operations.

Example:

10 → 20 → 30
↑ ↑
front rear

dequeue() removes 10.

3

Q27. How do you check for valid parentheses?

Push opening brackets onto a stack; for closing brackets, pop and check for a match (O(n)).

Example:

"({[]})"

This is valid.

But:

"([)]"

is invalid because the closing brackets don't match the required order.

Time:

O(n)

Q28. What is the Next Greater Element problem?

Find the first greater element to the right for each item using a monotonic stack for O(n) time.

Example:

Input:
[4, 5, 2, 10]

Output:
[5, 10, 10, -1]

Q29. What is a Min Stack?

A stack that tracks the minimum element in O(1) time by using an auxiliary stack to store current minimums.

Example:

push(5)
push(3)
push(7)
push(2)

Minimum:

2

After popping 2, minimum becomes:

3

Q30. What is the difference between a Binary Tree and a BST?

A Binary Tree has up to two children per node; a BST ensures Left < Node < Right ordering.

Example:

```
   8
  / \
 4   12
/ \
2  6
```

This is a BST because smaller values are on the left and larger values are on the right.

Q31. What is DFS vs BFS?

DFS (Stack/Recursion) explores deep first; BFS (Queue) explores level-by-level.

Example:

```
   1
  / \
 2   3
/ \
```

```
  4   5
```

DFS might visit:

1 → 2 → 4 → 5 → 3

BFS:

1 → 2 → 3 → 4 → 5

Q32. What are Preorder, Inorder and Postorder traversals?

Preorder (R-L-R), Inorder (L-R-R), and Postorder (L-R-R) are DFS traversals. BST Inorder is sorted.

Important interview point:
Inorder traversal of a BST produces values in sorted order.

Q33. What is level-order traversal?

Level-order traversal visits nodes level by level using a queue (BFS on trees).

Example:

```
    1
   / \
  2   3
 / \
4   5
```

Level order:

1 → 2 → 3 → 4 → 5

Q34. How do you find the height of a binary tree?

Recursively calculate height as 1 + max(leftHeight, rightHeight).

```
    1
   / \
  2   3
 /
4
```

Height = 3.

Interview tip:
Always clarify whether the interviewer measures height in **nodes or edges**, because both conventions exist.

Q35. How do you check whether a binary tree is balanced?

A tree is balanced if the height difference between subtrees for every node is 1.

Example:

```
    1
   / \
  2   3
 /
4
```

This can be balanced.

But:

```
1
 \
  2
   \
    3
     \
      4
```

is not balanced.

Q36. What is Lowest Common Ancestor?

The LCA is the deepest common ancestor of two nodes in a tree.

Example:

```
    1
   / \
  2   3
 / \
4   5
```

LCA of 4 and 5 is:

2

LCA of 4 and 3 is:

1

Q37. How do you validate a BST?

Recursively check if each node falls within a valid (min, max) range passed from ancestors.

Example:

```
  8
 / \
4   12
```

For node 4:

(-∞, 8)

For node 12:

(8, +∞)

This ensures the entire subtree follows BST ordering.

Q38. What is BFS?

BFS explores level-by-level using a queue; ideal for finding shortest paths in unweighted graphs.

Example:

```
A — B — D
|
C
```

Starting from A:

A → B → C → D

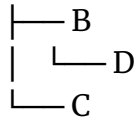
BFS is especially useful for finding the **shortest path in an unweighted graph**.

Q39. What is DFS?

DFS explores as deep as possible using recursion/stack; useful for connectivity and cycle detection.

Example:

A



DFS might visit:

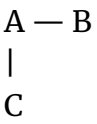
A → B → D → C

Q40. Adjacency Matrix vs Adjacency List?

Matrix uses O(V) space (better for dense graphs); List uses O(V+E) space (better for sparse graphs).

Example:

For:



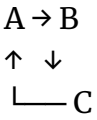
Adjacency list:

A → B, C
B → A
C → A

Q41. How do you detect a cycle in a graph?

Undirected: check for visited non-parents. Directed: check for nodes in the current recursion stack.

Example:



There is a cycle:

A → B → C → A

Interview tip:
Always ask:

> "Is the graph directed or undirected?"

The cycle-detection technique differs.

Q42. What are connected components?

Connected components are groups of vertices where every vertex is reachable from others in the same group.

Example:

A — B — C

D — E

F

There are:

3 connected components
{A,B,C}
{D,E}
{F}

Q43. What is the shortest path problem?

Find the path with minimum cost. Use BFS for unweighted, Dijkstra for non-negative weights, Bellman-Ford for negative weights.

- **Unweighted graph** → BFS
- **Non-negative weighted graph** → Dijkstra
- **Negative edge weights** → algorithms such as Bellman-Ford may be needed

Example:

A --2-- B
| |
5 1
| |
C --2-- D

The shortest path from A to D is:

A → B → D

Cost = 2 + 1 = 3

Q44. What is Dijkstra's Algorithm?

Greedily find shortest paths using a Priority Queue by relaxing edges for unvisited nodes with smallest distances.

1. Start with the source.
2. Choose the unvisited vertex with the smallest known distance.
3. Relax its neighboring edges.
4. Repeat.

Example:

A --2-- B --1-- D

Starting from A:

A → B = 2
A → B → D = 3

So shortest distance to D is 3.

Important:
Dijkstra's algorithm does **not** work correctly with negative edge weights.

Q45. What is topological sorting?

A linear ordering of vertices in a DAG where for every edge A B, A comes before B.

Example:

Suppose:

Learn Programming
↓
Learn DSA
↓
Solve Problems

A valid order is:

Programming → DSA → Problems

- DFS
- Kahn's Algorithm using indegrees and a queue

Important:
Topological sorting is possible only for a **directed acyclic graph**.

Q46. What is Dynamic Programming?

A technique to solve problems with overlapping subproblems by storing and reusing results (memoization/tabulation).

1. Defining a state.
2. Finding a recurrence/transition.
3. Defining base cases.
4. Storing results.

Example: Fibonacci

Naive recursion repeatedly calculates the same values:

```
fib(5)
├── fib(4)
│   ├── fib(3)
│   └── fib(2)
└── fib(3)  ← calculated again
```

DP stores the result of fib(3) instead of calculating it again.

Q47. What is memoization vs tabulation?

Memoization: Top-down + recursion + storage. Tabulation: Bottom-up + iteration + table.

Example: Fibonacci

Memoization:

```
fib(5)
↓
fib(4)
↓
fib(3)
↓
store results
```

Tabulation:

```
fib(0)
fib(1)
↓
fib(2)
↓
fib(3)
↓
fib(4)
↓
fib(5)
```

Easy way to remember:

Memoization → Top-down + recursion
Tabulation → Bottom-up + iteration

Q48. What is the 0/1 Knapsack problem?

Maximize total value within a weight capacity by deciding to either take or leave each item (binary choice).

weight
value

and a bag with a maximum capacity.

Take it
OR
Don't take it

That's why it's called **0/1** knapsack.

Example:

Capacity = 5

Item A: weight 2, value 10
Item B: weight 3, value 15
Item C: weight 4, value 20

Taking A + B gives:

Weight = 5
Value = 25

So the maximum may be 25 for this example.

Q49. What is the Climbing Stairs problem?

Find ways to reach step n using 1 or 2 steps; follows the recurrence ways(n) = ways(n-1) + ways(n-2).

ways(n) = ways(n-1) + ways(n-2)

Example:

For 3 stairs:

1 + 1 + 1
1 + 2

2 + 1

So there are:

3 ways

Q50. What is the Coin Change problem?

Find the minimum number of coins needed to make a target amount using DP to build from smaller subproblems.

Example:

Coins = [1, 2, 5]
Amount = 11

The minimum number of coins is:

5 + 5 + 1 = 11

So:

3 coins

Q51. What is Longest Common Subsequence (LCS)?

LCS is the longest sequence present in both strings in the same relative order (not necessarily contiguous).

Example:

String 1 = "abcde"
String 2 = "ace"

The LCS is:

"ace"

Length:

3

Q52. What is Longest Increasing Subsequence (LIS)?

LIS is the longest subsequence where elements are in strictly increasing order (not necessarily contiguous).

Example:

[10, 9, 2, 5, 3, 7, 101, 18]

One LIS is:

[2, 3, 7, 101]

Length:

4